

进程间通信

进程间通信（Inter-Process Communication，IPC）是多进程协作的基础。通过多进程协作来实现应用和系统是一种被广泛使用的开发方法，如类 UNIX 系统中使用许多功能独立且完整的程序组合来完成复杂的任务。相比用单个进程来实现所有功能的方式，多个实现不同功能的进程彼此协作的方式主要具有以下三点优势。

- 将功能模块化，避免重复造轮子。例如，在智能手机平台，画面绘制是大部分应用的共性功能。为了避免重复实现，Android 提供了一系列服务进程来专门负责绘制相关的任务，应用只需要通过进程间通信机制将需要绘制的画面信息传递给绘制进程，调用相应的服务即可，这样大大简化了应用的开发。此外，由于进程本身的独立性，相比其他如基于库的模块化方法，多进程协作方式的接口更加明确，并且能够更好地“解耦”开发人员。
- 增强模块间的隔离，提供更强的安全保障。把应用的所有模块都放在一个进程里，虽然内部交互较为方便，但是隔离性相对较弱，只要攻击者控制其中任何一个模块，就可以进而攻击其他模块。相反，将应用的模块分别部署到多个互相隔离的进程里，即使某个进程被攻击者控制，也只能危害这一个进程中的计算任务，其他进程在做好接口检查的前提下，仍然可能保证敏感代码和数据的安全。当前的浏览器系统中通常会采用类似的技术来保障不同的页面之间的隔离性。
- 提高应用的容错能力。和隔离性类似，如果应用的所有数据和代码都在同一个进程地址空间内，那么任何一个运行错误（如除零错误）都可能导致整个应用的崩溃。意外的崩溃不仅会破坏用户体验，还可能导致重要数据的丢失。

本章将介绍进程间通信的基础概念和特性，以及一些经典的进程间通信设计。

8.1 进程间通信基础

本节主要知识点

- 进程间通信有哪些设计选择？
- 进程间通信过程中，数据和控制流分别是怎么传输和转移的？
- 进程间通信的连接是怎么建立的？
- 什么是超时机制？为什么进程间通信需要引入超时？
- 一个进程如何找到另一个进程提供的服务？

一般而言，进程间通信至少需要两方（如两个进程）参与。根据信息流动的方向，这两方通常被称为发送者和接收者。在实际使用中，IPC 经常被用于服务调用，因此参与 IPC 的两方又被称为调用者和被调用者，或者客户端和服务端。后文会根据具体场景使用不同的术语。在具体的系统中，通常还会存在多方通信的情况，如多个发送者单个接收者（ $N:1$ ）、单个发送者多个接收者（ $1:M$ ）或多个发送者多个接收者（ $N:M$ ）。

8.1.1 进程间通信接口

从操作系统抽象看进程间通信。前面章节对虚拟内存和进程进行了介绍，其中，虚拟内存能够保障两个进程间的内存隔离。然而，虚拟内存的隔离性同时也限制了两个进程间的直接交互与通信。为了交互，必然需要开启一个信道。这个信道的设计有多种选择：从内核是否介入的角度，信道可以在用户态（内核不介入或很少介入，如共享内存），也可以在内核态（内核介入，如管道）；从操作系统接口的角度，信道可以是内存接口（如共享内存），也可以是文件接口（如管道），甚至是新的接口（如消息接口等）。表 8.1 展示了现有的一些进程间通信机制[⊖]及其接口。

表 8.1 进程间通信机制及其接口。其中，简单 IPC 为本章（8.1.2 节）介绍的基于共享内存的进程间通信方案。共享内存的进程间通信虽然需要内核来建立共享内存区域，但是主要的通信方法通常是由用户态实现的，因此本表使用“用户态”作为其实现方式

	通信接口	实现方式	方向	通知方式
简单 IPC	消息接口	用户态	双向	轮询
共享内存	内存接口	用户态	均可	轮询
管道	文件接口	内核态	单向	等待 + 内核唤醒
信号	信号接口	内核态	单向	内核回调
消息队列	消息接口	内核态	均可	轮询

⊖ 除了信号机制外，其余机制在本章中均会介绍。

消息接口。除了传统的内存、文件等接口之外，操作系统通常还会提供消息接口用于进程间通信。消息接口将数据抽象成一个个的消息在两个（或多个）进程间传递。需要注意，不同的 IPC 设计可以有不同的消息抽象，如后文将介绍的消息队列中带类型的消息结构体抽象。

消息接口通常包括：

- 发送消息：Send(message)。
- 接收消息：Recv(message)。
- 远程方法调用：RPC(req_message, resp_message)。
- 回复消息：Reply(resp_message)。

此外，这里的参数只关注消息，实际的接口中通常包含其他选项。其中，发送和接收消息的接口语义较为直接。如代码片段 8.1 和代码片段 8.2 所示，发送者可以通过 Send 将一个消息发送给接收者，而接收者会使用 Recv 来接收该消息。需要额外注意的是，在这个代码片段中，发送者和消费者需要依赖于通信连接 chan，将其作为一个媒介进行消息的传输。

代码片段 8.1 消息的发送 (Send)

```
1 // IPC 的发送者
2 int main(void)
3 {
4     Message msg;
5     // chan 表示发送者和消费者之间的“通信连接”
6     Channel chan = simple_ipc_channel(...);
7     // 按照语义生成请求消息
8     msg = construct_request(...);
9
10    // 通过通信连接发送消息
11    Send(chan, &msg);
12    ...
13 }
```

代码片段 8.2 消息的接收 (Recv)

```
1 // IPC 的接收者
2 int main(void)
3 {
4     Message msg;
5     // chan 表示发送者和消费者之间的“通信连接”
6     Channel chan = simple_ipc_channel(...);
7
8     while (1) {
9         // 等待一个消息的到来，这里会收到 Send 发送的消息
10        Recv(chan, &msg);
11        // 处理消息
```

```
12     results = handle_msg(&msg);
13     ...
14 }
15 ...
16 }
```

而远程方法调用（Remote Procedure Call, RPC）通常可以理解成（发送端）调用 Send 接口后紧接着调用 Recv 接口，相当于在发送消息（对应 req_message）之后，发送端还会等待一个返回消息（对应 resp_message）。具体的区别在关于“IPC 方向性”（8.1.5 节）的讨论中会进一步说明。Reply 通常用作回复远程方法调用。代码片段 8.3 和代码片段 8.4 给出了一个类似的例子。在这个例子中，发送者通过 RPC 接口向接收者发送了一个请求消息（req_msg），并等待结果消息（resp_msg）的返回。而接收者会首先通过 Recv 接口接收消息，然后对其进行处理，并且在最后通过 Reply 接口返回结果消息给接收者，从而完成一次进程间通信。

代码片段 8.3 消息的远程方法调用：发送者

```
1 #include <simple-ipc.h> // 使用后续章节的简单 IPC 设计
2 ...
3
4 // IPC 的发送者
5 int main(void)
6 {
7     Message req_msg, resp_msg;
8     // chan 表示发送者和消费者之间的“通信连接”
9     Channel chan = simple_ipc_channel(...);
10    // 按照语义生成请求消息
11    req_msg = construct_request(...);
12
13    // 以 RPC 的方式调用接收者，并阻塞等待结果的返回
14    RPC(chan, &req_msg, &resp_msg);
15
16    printf("The response is: %s", msg_to_str(resp_msg));
17    ...
18 }
```

代码片段 8.4 消息的远程方法调用：接收者

```
1 #include <simple-ipc.h> // 使用后续章节的简单 IPC 设计
2 ...
3
4 // IPC 的接收者
5 int main(void)
6 {
7     Message req_msg, resp_msg;
8     // chan 表示发送者和消费者之间的“通信连接”
9     Channel chan = simple_ipc_channel(...);
```

```
10
11 while (1) {
12     // 等待消息的到来
13     Recv(chan, &req_msg);
14     // 处理消息并构建结果消息
15     resp_msg = handle_msg(&req_msg);
16     Reply(chan, &resp_msg);
17 }
18 ..
19 }
```

需要注意的是，这些消息接口可以是操作系统提供的，也可以是用户态库封装提供的。在 RPC 的案例中，我们使用的是下一节即将介绍的简单 IPC 的进程间通信设计。

练习

8.1 远程方法调用和消息发送的相同点与不同点分别是什么？

8.1.2 一个简单的进程间通信设计

本节通过一个简单的例子来介绍 IPC 中的基本概念。

本节中的简单 IPC 只涉及两个进程：发送者进程和接收者进程。通信的过程由发送者发起，将一段定长的数据发送给接收者，之后发送者会等待返回数据（即 RPC 接口）。接收者接收到数据后，会处理数据并将结果返回给发送者（即 Recv 和 Reply 接口）。整个通信的过程从发送者发起通信开始，到发送者收到返回结果结束。

IPC 通常包含两个阶段：准备阶段和通信阶段。准备阶段需要在通信的进程间建立一个通信连接，这个通信连接是打通两个隔离的进程的信道。而通信阶段通常可以进一步划分为数据传递和通知机制。简单 IPC 设计中选择使用消息接口作为通信接口。

通信连接。不同的进程处在不同的虚拟地址空间内，可访问的内存是互相隔离的。为了实现一个简单的 IPC，此处假设内核已为两个进程映射了一段共享内存，如图 8.1 所示。即在两个进程的地址空间中，分别有一段虚拟地址区间映射到同一段物理内存^①。共享内存打破了进程之间的地址空间隔离，使两个进程有了一个可以交换数据的缓冲区，可通过这个缓冲区进行通信。需要注意的是，对两个进程来说这块共享内存所映射的虚拟地址可以是不同的。基于共享内存，简单 IPC 方案的通信连接是在建立共享区域的一瞬间完成的。

^① 共享内存可以支持多于两个进程，此处只考虑两个进程的情况。

回顾一下代码片段 8.4 中的 `simple_ipc_channel` 方法，它其实就是在建立共享内存，从而建立通信连接。

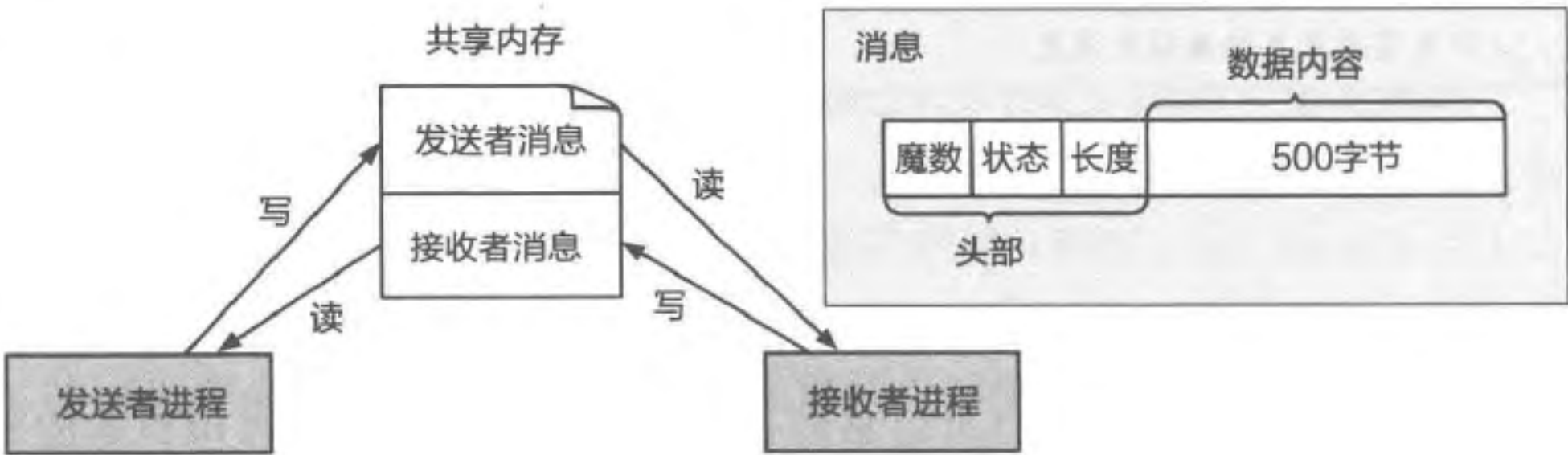


图 8.1 简单 IPC 设计

数据传递。由于简单 IPC 使用消息接口，其通信数据抽象是消息（Message）。发送者将数据以消息的格式传递给接收者，接收者将结果数据同样以消息的格式返回给发送者。消息的格式实现如代码片段 8.5 所示。消息一般包含一个头部（Header）和一段数据内容（Payload）。头部中通常会包含魔数（Magic Number）、长度、状态等信息。数据内容中既可以包含“纯”数据（如字符串），也可以包含系统资源（如文件描述符）。在简单 IPC 方案中，消息的数据内容仅包含“纯”数据，长度固定为 500 字节。

代码片段 8.5 简单 IPC 设计：数据结构

```
1 #define BUFFER_SIZE 2
2 #define PAYLOAD_SIZE 500
3 typedef struct {
4     int magic;           // 头部中的魔数
5     int status;          // 头部中的状态
6     int length;          // 头部中的长度
7 } Header;
8
9 typedef struct {
10     Header header;       // 头部
11     char payload[PAYLOAD_SIZE]; // 数据内容
12 } Message;
13
14 Message buffer[BUFFER_SIZE]; // 共享内存区域
15 int sender_idx = 0;          // 发送者消息对应的 Buffer 中的序号
16 int receiver_idx = 1;        // 接收者消息对应的 Buffer 中的序号
```

通信过程与通知机制。通信过程一般指通信的进程间具体的通信发起、回复等过程。在简单 IPC 设计中，假设共享内存刚好可以存放两个消息，如图 8.1 所示。其中，一个消息由发送者用来发起通信（称为发送者消息），另一个消息由接收者用来响应通信和发

送返回结果（称为接收者消息）。开始时，两个消息的状态都是无效的（见消息头部的“状态”），接收者不停地轮询发送者消息的状态信息。当开始 IPC 时，发送者会将要传输的数据内容拷贝到发送者消息上，然后依次设置消息的头部字段，最后一步是将状态设置为“准备就绪”，之后开始轮询接收者消息的状态。一直在轮询的接收者一旦观测到发送者信息的状态为“准备就绪”，就表示发送者发出了一个消息。接收者在读取发送者的消息后，将发送者消息的状态设置为“无效”。此后，接收者根据消息中的数据处理请求，将返回数据写入接收者消息，设置消息的头部字段，最后将消息状态设置为“准备就绪”。当发送者观测到接收者消息的状态为“准备就绪”后，即表示收到了返回的结果。类似地，发送者读取消息后，需要将接收者消息的状态设置为“无效”。这就完成了一次简单 IPC 的通信发起和回复过程。在这个过程中，简单 IPC 采用轮询的方式作为通知机制。

这个方案看似简单，但其中有很多问题值得深入思考，如通信带宽和时延表现如何？如何优化带宽和时延性能？消息数据段设置过长会带来内存浪费问题，过短则会导致频繁的拆分和通信，如何解决这个问题？对消息的轮询会导致 CPU 资源的大量浪费，这样的开销在通信不频繁的场景下格外突出，是否有更高效的通知方式？对于这些问题的讨论，本章的后续内容会一一展开。

练习

- 8.2 简单 IPC 中，发送者在发送完消息后，需要轮询等待一个接收者的消息，为什么？
- 8.3 简单 IPC 中，为什么接收者在读取发送者消息后，需要将发送者消息的状态设置为“无效”？可以由发送者来做这一步操作吗？

8.1.3 数据传递

从简单 IPC 方案中可以看出，进程间通信的一个重要功能是在进程间传递数据。数据传递往往需要一个信箱或媒介来完成，下面将分别介绍以共享内存或操作系统为媒介的数据传递方式。

基于共享内存的数据传递

在简单 IPC 方案中，我们已经看到共享内存是如何帮助两个进程进行通信的。基于共享内存的数据传递的一个特点是：操作系统在通信过程中不干预数据传输。从操作系统的角度来看，共享内存为两个（或多个）进程在它们的虚拟地址空间中映射同一段物理内存，这些进程基于共享内存实现具体的通信方案，操作系统通常不参与后续的数据传输过程。

小知识：序列化与反序列化

由于虚拟内存的存在，进程间通信的双方所映射的共享内存往往有着不同的虚拟地址。一旦通信的一方在共享内存区域中写入一些较为复杂的结构（如本地指针），另一方是无法直接使用这些复杂结构的。一个简单的例子是双向链表，该链表中的每一个元素都包含指向前一个和后一个元素的指针。当发送者进程直接将这样一个链表写入共享内存区域后，由于两端的虚拟地址不同，接收者是无法准确知道读取出来的指针的具体含义的。甚至如果发送者是一个恶意的进程，接收者在不设防的情况下使用对应的指针，则会导致一些攻击的发生。

为了保证通过共享内存传递的“消息”在通信的双方有相同的语义，一些 IPC 设计通常会引入序列化和反序列化的操作。进程间通信过程中，序列化能够将复杂的结构转化为便于传输的格式，如将上述链表中的指针地址转化为一个相对的偏移量。反序列化是序列化的逆过程，该过程会将传输的格式转化为进程实际使用的结构，如接收者进程能够利用接收到的链表数据和其中的偏移量，重新构造对应的元素之间的指针。

操作系统辅助的数据传递

操作系统辅助的数据传递指内核为用户态提供通信的接口，如 `Send` 和 `Recv` 等，进程可以直接使用这些接口，将数据传递给另一个进程，而不需要建立共享内存和轮询内存数据等操作。

以前面介绍的消息接口为例，当进程 P 和进程 Q 希望通过内核接口进行通信时，它们通常需要：通过特定内核接口建立通信连接；通过 `Send`、`Recv` 接口进行通信，并且在通信过程中通过内核传递数据^①。这里建立通信连接的过程，和通过内核建立共享内存区域是类似的。只是这里更多地强调抽象意义上的连接建立，如内核可以维护一个数据结构来记录建立好连接的进程对。连接建立的细节会在后面通过单独的小节进行介绍（8.1.8 节）。

共享内存和操作系统辅助传递的对比

共享内存和操作系统辅助传递这两种方式各有优劣，在实际使用中通常需要结合具体的场景和需求进行选择。从数据传递的性能来看，使用共享内存的方式，可以实现理论上零内存拷贝的传输，比如前文介绍的简单 IPC。使用操作系统辅助传递的方式，则通常需要两次内存拷贝：一次是将数据从发送者用户态内存拷贝到内核内存，另一次是从内核内存拷贝到接收者用户态内存。不过，两次拷贝并非绝对必要的，如后文中介绍的 L4 微内核系统^[1-4]结合内存重映射（Memory Remapping）技术，就能

① 注意，`Send` 等接口的底层实现既可以是共享内存，也可以是操作系统提供的底层接口（如文件接口），这里考虑的是操作系统提供的接口的情况。

够做到一次拷贝。

操作系统辅助传递同样有优于共享内存的地方。第一，操作系统辅助传递的抽象更简单。内核可以保证每一次通信接口的调用（这里假设是消息接口）都是一个消息被发送或接收（或者出现异常错误），并且能够较好地支持变长的消息，而共享内存则需要用户态软件封装来实现这一点。第二，操作系统辅助传递的安全性保证通常更强，并且不会破坏发送者和接收者进程的内存隔离性。第三，在多方通信时，在多个进程间共享内存区域是复杂且不安全的，而操作系统辅助传递可以避免此问题。

练习

8.4 在涉及多个进程的通信场景下，在所有进程间共享一个内存区域是不安全的。请思考怎么基于共享内存实现多个进程间的通信。

8.1.4 通知机制

实现进程间通信除了考虑数据传递外，往往还会附带通知机制：在新的数据（或消息）到来时通知通信的接收方。

在前文介绍的简单 IPC 方案中，进程依赖于轮询内存数据来检查是否有消息到来，而这可能会浪费大量系统 CPU 计算资源。为了避免 CPU 资源的浪费，操作系统支持的 IPC 方案中，内核通常会基于控制流转移来实现通知机制，如内核可以将控制流从发送者进程切换到接收者进程来实现通知接收者的效果（返回的过程类似）。

在关于调度的章节中，我们了解到用户态进程其实是运行在操作系统抽象出来的时间片上的，并且进程可以有多种运行状态，如运行中、阻塞中（不再被调度）。IPC 中的控制流转移，通常是利用内核对于进程的运行状态和运行时间的控制来实现的。常见的过程如图 8.2 所示。以基于消息接口的 IPC 方案为例，首先，接收者进程完成初始化后将自己阻塞起来等待消息的到来（如执行阻塞的 `Recv`），之后，发送者进程发起通信（`RPC`）。在处理该操作时，内核首先将发送者发送的消息传递给接收者，然后让发送者进程进入阻塞状态（等待接收者进程的回复消息），并将接收者进程从阻塞状态唤醒到可运行状态。对接收者进程而言，会看到阻塞的 `Recv` 返回了一个消息，表明接收到了来自发送者的消息。这个特定的通信过程展示了控制流是如何在进程间转移的。可以看到，结合内核中的调度以及对进程（或线程）的调度状态的修改，控制流转移可以避免轮询操作，高效地将消息的到来和发出“告知”进程。

8.1.5 单向和双向

简单 IPC 方案中的发送者进程会等待一个返回结果，即一次完整的通信过程包含两

个方向的通信：发送者传递一个消息（包含对应的请求）给接收者，以及接收者返回一个消息（包含请求的结果）给发送者。但是两个方向的通信显然不是绝对的。这就引出了通信中的又一个概念，即通信的方向性。IPC 通常包含三种可能的方向：仅支持单向通信、仅支持双向通信、单向和双向通信均可。三者的对比如图 8.3 所示。

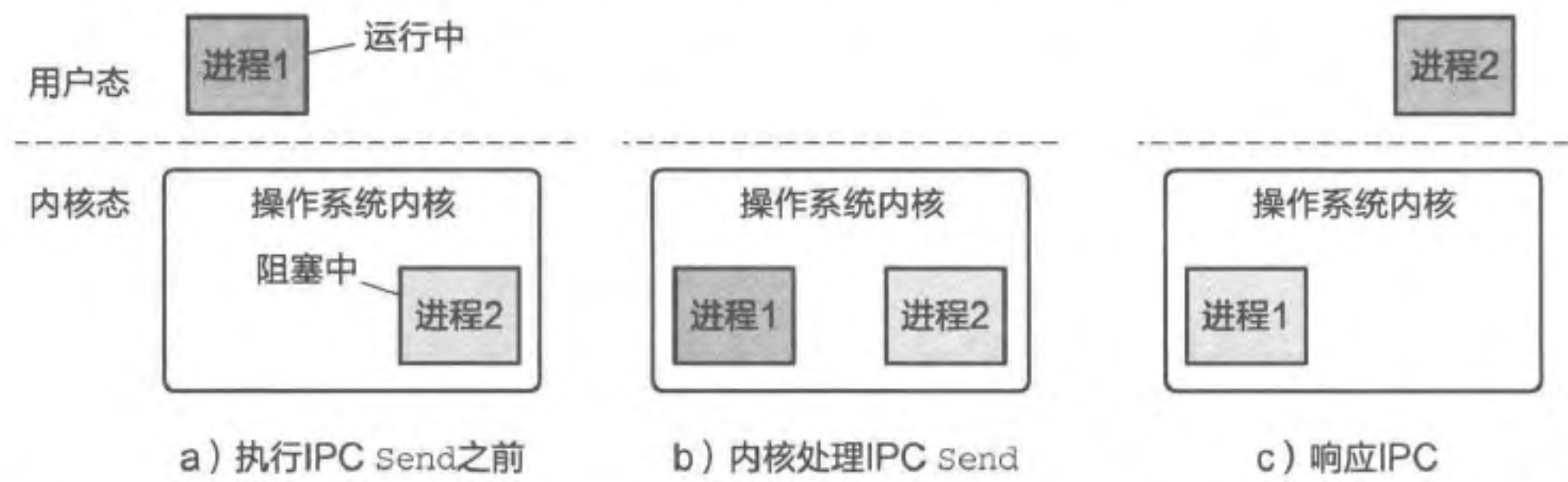


图 8.2 进程间通信中的控制流转移。深色表示进程正在用户态运行，浅色表示进程处于阻塞中，IPC Send 为 RPC 过程中的 Send 阶段。a) 进程 2 处于阻塞状态，等待新的请求到来；b) 进程 1 发起 IPC 请求，陷入内核处理；c) 内核将进程 2 唤醒去处理请求，而进程 1 处于阻塞状态等待执行结果

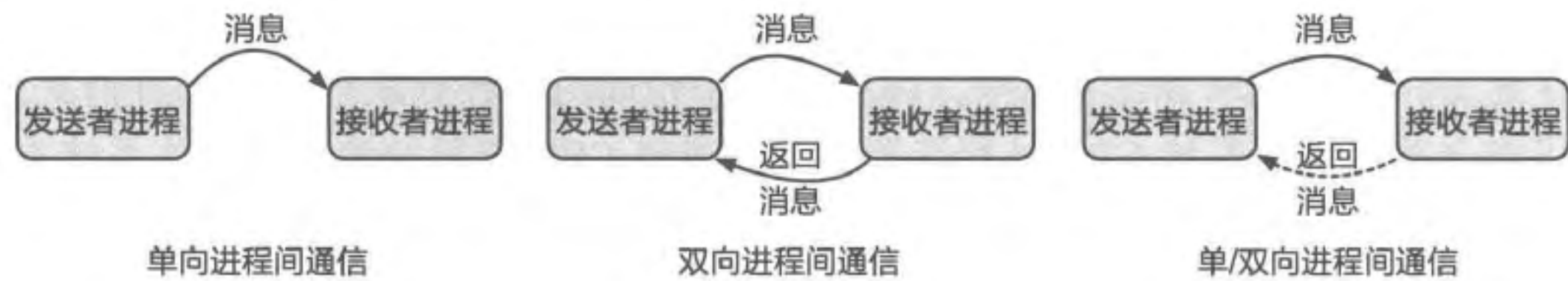


图 8.3 IPC 的方向

从图中可以看到，单向通信通常指消息在一个连接上只能从一端发送到另一端，双向则允许双方互相发送消息，而单 / 双向均可的方式则会根据通信中具体的配置选项等来判断是否需要支持单向或双向的通信。通常来说，单向通信其实是系统软件实现 IPC 的一个基本单元，双向通信是可以基于单向 IPC 来搭建的。在接口上，如果通信的两端在连接建立后分别只能使用 Send（发送消息）及 Recv（接收消息），那么这通常对应单向通信。而前文介绍的 RPC 接口则是一个有代表性的双向通信的例子。RPC 要求接收者处理好发送者发送的消息后返回一个消息（结果），从而完成整个通信，即这个过程中会涉及两次单向通信。在实际应用中，很多系统选择的是单 / 双向均可的策略，这样可以比较好地支持各种场景。当然，管道等只支持单向通信的机制在实际中同样有较多的应用。

8.1.6 同步和异步

进程间通信的另一种分类是：同步 IPC 和异步 IPC。简单来看，同步 IPC 指 IPC 操

作（如 send）会阻塞进程直到该操作完成；而异步 IPC 则通常是非阻塞的，进程只要发起一次操作即可返回，不需要等待其完成。同步 IPC 的 RPC 调用如图 8.4a 所示，可以将其看成一个线性的控制流：调用者发起 RPC 请求，然后控制流切换到被调用者。被调用者处理请求时，调用者处于阻塞状态。当被调用者执行完任务后，控制流会切回调用者中。调用者得到返回的结果后才可以继续执行。从调用者的角度来看，当 RPC 返回后，请求消息的发送和结果的接收都已经完成了。与之相对，异步 IPC 是多个并行的控制流，如图 8.4b 所示，当调用者发起 IPC 后，被调用者接收到通信的数据和请求后开始响应。同时，调用者的 IPC 调用不会等待被调用者执行，而是直接返回。异步 IPC 通常通过轮询内存状态或注册回调函数（如果内核支持）来获取返回结果。

同步 IPC 往往是双向 IPC（或 RPC），即发送者需要等待返回结果。不过也存在单向 IPC 是同步的，在这种场景下，虽然发送者不会阻塞等待接收返回结果，但是发送者会阻塞等待接收者接收。考虑一个具体的场景，假设操作系统支持多方通信（暂不深入解释多方和双方的区别）：允许一个连接上有多个发送者，但不支持多个接收者。这种场景下，一个可能的情况是，发送者发送消息的时候接收者正在接收和处理其他发送者的消息。在同步的设计下，此时发送者需要等待一定的时间，而异步的设计则会通过内核缓冲区等方式暂存消息，从而避免等待。

在早期的微内核系统（如 L4 微内核）中，同步 IPC 往往是唯一的 IPC 方式。这是因为相比异步而言，同步 IPC 有着更好的编程抽象。使用（同步的）RPC 时，调用者可以将进程间通信看成一种“函数调用”，调用返回也就意味着结果返回了，主要的不同是在 IPC 的场景下执行函数的可能是另外一个进程。

然而同步 IPC 在操作系统的发展中逐渐表现出一些不足之处，一个典型的问题是并发。当一个服务进程要响应很多客户进程的通信时，比如一个微内核中的用户态文件系统，在同步 IPC 的实现下，服务进程（为了性能）往往需要创建大量工作线程去响应不同的客户进程，否则有可能出现阻塞客户请求的情况。然而，这往往需要权衡，即过少的工作线程会导致大量客户进程被阻塞，过多的工作线程会浪费系统资源。而使用异步 IPC 则可以在并发通信时避免这类问题。后续的一些设计（如 Android Binder）通过线程池的模型克服了同步 IPC 下的并发挑战。总的来看，目前大部分操作系统内核都会选择同时实现同步和异步 IPC 来满足不同的应用需求。

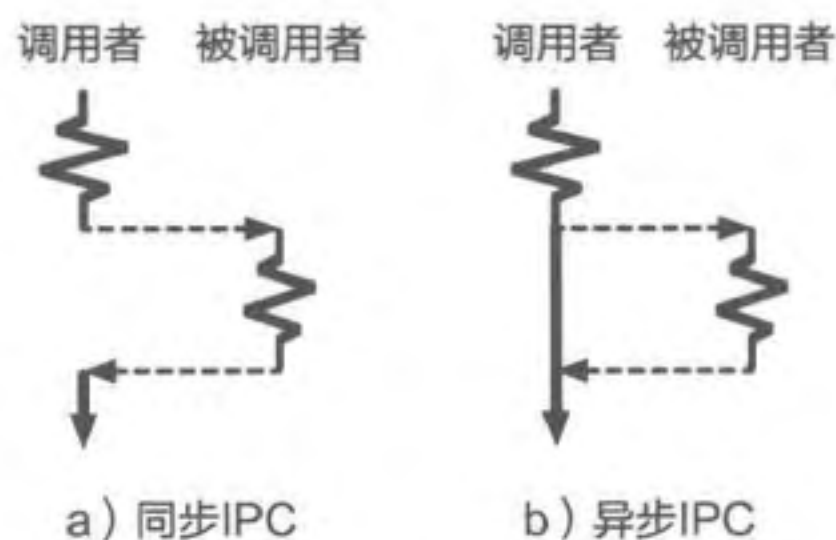


图 8.4 同步和异步 IPC

8.1.7 超时机制

进程间的隔离性为通信带来的一个问题是：通信的双方很难确认对方的状态。以同

步 IPC 为例，当调用者切换到被调用者的控制流后，如果被调用者恶意地不返回到调用者，那么就会使得调用者无法继续执行。异步 IPC 也有类似的问题。即使控制流不会被被调用者恶意抢占，调用者仍然有可能花费过长的时间来等待一个请求的处理。实际上，即使被调用者并非恶意的，也有可能因为调度、被调用者过于忙碌（需要处理大量其他请求）或者某些错误导致一些请求丢失，从而出现上述问题。

为了解决这个问题，IPC 的设计中引入了超时机制。超时机制扩展了 IPC 通信双方的接口，允许调用者 / 接收者指定它们发送 / 接收请求的等待时间。比如，一个应用程序可以花费 5 秒等待文件系统进程的 IPC 请求处理操作。如果超过 5 秒仍然没有反馈，则由 OS 内核结束这次 IPC 调用，返回一个超时的错误。超时机制允许进程对于一次通信的等待时间设置上限，从而避免类似拒绝服务（Denial-of-Service）的攻击情况出现。这似乎是一个很完美的解决方案。

然而在实际情况中，大部分进程很难决定合理的超时时间。例如前文 5 秒超时的例子，当进程间传输的数据量十分大，或者文件系统（作为接收者进程）本身无法获得足够的时间片来处理请求时，就可能无法及时完成任务。定义过短的超时可能会导致调用者频繁地重试某一个 IPC 调用，而定义过长的超时则可能无法及时察觉被调用者的异常。因此，目前内核常常引入两个特殊的超时选择：“阻塞”和“立即返回”。“阻塞”其实和引入超时之前的机制是类似的；而“立即返回”则意味着只有当前被调用者处于可以立即响应的状态时才会真的发起通信，否则就直接返回。发送者进程会根据需求进行选择，更加注重安全性的往往选择“立即返回”，而更加注重功能性的则倾向于选择“阻塞”。然而，即使超时机制存在这些问题，目前仍然鲜有更好的选择作为替代。

8.1.8 通信连接

通信连接的建立在前文中已多次提到。对于基于共享内存的进程间通信方案，通信连接的建立通常是在建立共享区域的一瞬间完成的。而对于涉及内核的控制流转移的通信而言，通信连接管理是内核 IPC 模块中很重要的一部分。

通信连接管理和前文介绍的 IPC 的各种概念是息息相关的。比如，连接如果能够用于多于两个的进程，就意味着该 IPC 设计是支持多方通信的。而连接能否缓冲数据（消息）在一定程度上决定了 IPC 设计是异步还是同步的。连接本身的单向和双向则直接关联着 IPC 的方向性。

本节将回答“如何建立连接”。虽然实际系统中有各种不同的实现，但它们大部分可以被归为两类对应的设计——直接通信和间接通信。

直接通信是指通信的进程一方需要显式地标识另一方。以 `Send` 和 `Recv` 为例，需要将它们完善成：发送消息，`Send(P, message)` 给进程 `P` 发送一个消息；接收消

息, `Recv(Q, message)` 从进程 Q 处接收一个消息。直接通信下连接的建立是自动的, 在具体交互时通过标识的名字完成。这也就意味着一个连接会唯一地对应一对进程, 而一对进程之间也只存在一个连接^①。连接本身既可以是单向的, 又可以是双向的(更常见)。

与之相对, 间接通信需要经过一个中间信箱来完成通信。每个信箱有自己唯一的标识符, 而进程间通过共享信箱来交换消息, 即进程间连接的建立发生在共享信箱时。每对进程可以通过共享多个信箱的方式来建立多个连接, 连接同样可以是单向的或双向的。

在本章后续介绍的具体案例中, 两类方式都有系统在使用。例如, 管道就属于间接通信方式, 通信的双方进程并不知道对面是哪一个特定的进程, 它们只知道双方共享这一管道, 管道本身肩负着类似信箱的任务。

练习

8.5 请思考简单 IPC 方案归在哪一类通信(直接或间接)更合理。为什么?

8.1.9 权限检查

进程间的通信通常依赖于一套权限检查机制来保证连接的安全性。微内核系统中常用基于 Capability^[5] 的安全检查机制, 而在如 Linux 这样的宏内核系统中, 通常会将安全检查机制和文件的权限检查结合在一起处理。

微内核中的进程间通信大部分是涉及多方的, 不过通常是多发送者、单一接收者。这种多方的通信场景其实不难想象, 比如用户态文件系统。各个使用文件系统服务的进程都需要和文件系统建立连接从而进行通信。那么, 问题是: 怎么判断哪些进程是可以连接到这个文件系统的呢? 这个问题对于一些私有的服务端进程(如某个特定应用的数据库进程)更加重要。如果任意进程都可以连接到私有的服务端进程, 那么显然会出现安全性的问题。

使用权限检查能够解决这个问题。一个进程要与其他进程通信, 必须通过权限检查。例如, `seL4`^[6] 等微内核系统中的 Capability 机制, 会将所有的通信连接抽象成一个内核对象, 而每个进程对于内核对象的访问权限(以及能够在该内核对象上执行的操作)由 Capability 来刻画。当一个进程企图和某其他进程通信时, 内核会检查该进程是否拥有 Capability, 是否有足够的权限访问一个连接对象并且对象是指向目标进程的。类似地, 宏内核(如 Linux 系统)通常会复用其有效用户/有效组的文件的权限, 从而刻画进

^① 为简化讨论, 此处未考虑多线程进程。在多线程场景下, 连接可以使用线程标识符。

程对于某个连接的权限。

System V 进程间通信权限管理

System V 进程间通信通常指宏内核下三种具体的进程间通信机制，即 System V 消息队列、System V 信号量和 System V 共享内存。在 Linux 等系统中，这三种机制通常共享一套权限管理方法。

在 Linux 等宏内核系统中，System V 通信的权限管理基于文件的权限检查机制。权限检查机制通常依赖于进程的用户分类（所有者用户 / 用户组用户 / 其他用户），并且是基于文件的权限抽象（可读 / 可写 / 可执行）来判定的。每个文件都会存储一个根据三类用户以及三种可能的权限组合而成的访问模式（Mode）。进程访问（如读操作）文件时，系统内核会判断这个进程是不是该文件的所有者（Owner），或者是否可以访问该文件的组（Group）中的成员，然后再根据其用户分类检查对应分类下的权限。

将上述权限检查机制应用到进程间通信场景时，内核会将通信连接抽象成一个个具体对象，比如对消息队列而言，在内核中会有多个消息队列的对象存在。在通信连接上的操作，如发送消息、接收消息，都会被抽象成对内核对象的访问行为。如在消息队列的设计中，发送消息相当于对消息队列对象的写操作，而接收消息相当于对消息队列的读操作。这样一来，只要 Linux 内核能够为每个通信的对象准备一个类似文件中的“访问模式”，就可以根据它来检查进程的操作是否合法。这个“访问模式”的内容，就是 Linux 内核中负责进程间通信权限管理的 IPC_PERM 结构。

IPC_PERM 中的内容如代码片段 8.6 所示。整个结构体的内容其实很简单，可以从名字推测出大部分基本含义。主要内容是所有者、创建者的用户和组标识符，以及访问模式。和检查文件访问权限的操作相同，内核会根据用户进程的用户和组信息，再结合内核对象中的信息，对操作进行检查。此外，该结构体中的 key 是 System V IPC 对象的标识符。进程可以根据 key 来索引 IPC 对象（或 IPC 连接）。

代码片段 8.6 System V 通信权限管理基于 IPC_PERM（基于 Linux 5 内核简化）

```
1 struct ipc_perm {
2     key_t key;
3     uid_t uid;    // 所有者的 uid 和 gid
4     gid_t gid;
5     uid_t cuid;   // 创建者的 uid 和 gid
6     gid_t cgid;
7     mode_t mode;  // 访问模式
8 };
```

8.1.10 命名服务

权限检查机制的引入保证了安全性。但另一个问题是，如何分发权限呢？通常，权

限的分发会通过一个用户态的服务——命名服务（Naming Service）——来处理。命名服务进程则指代提供该服务的具体进程。

什么是命名服务呢？命名服务像是一个全局的看板，可以协调服务端进程和客户端进程之间的信息。简单来说，服务端进程可以将自己提供的服务告诉命名服务进程，比如文件系统进程可以注册一个“文件系统服务”，网络系统进程可以注册一个“网络服务”。而客户端进程可以在命名服务中查询当前服务，并选择自己希望建立连接的服务去尝试获取权限。具体是否分发权限给客户端进程，是由命名服务和对应的服务端进程根据特定的策略来判断的。比如，文件系统进程的策略可能是，允许命名服务将连接文件系统的权限任意分发，这样所有的进程都可以使用全局的文件系统。而数据库进程的策略是，客户端必须提供特定私钥签名的证书，并且证书符合数据库进程的要求，才能完成建立。使用命名服务有很多好处，比如各个服务不再是内核中的 ID 等抽象的表示，而是对应用更友好的“名字”。前面章节中提到过的“服务发现”，通常就是使用命名服务来实现的。

为什么命名服务一般在用户态？从上面的介绍中可以看到命名服务的功能其实并不简单，并且通常需要支持很多不同的策略。将这些策略实现在内核中，对于微内核系统显然是不合适的。而即使是宏内核系统，这也不是一种优雅的设计。举例来看，宏内核的 Android Binder、ROS 以及微内核的 L4 等，都是依赖（用户态的）命名服务来完成连接权限分发的。当然，在具体的设计细节上，它们仍然存在很多的不同。

除了命名服务外，另外一种常见的方式是通过继承来分发连接权限。比如在 Linux 中，匿名管道通常用于父子进程之间的通信，内核通过在 fork 操作的时候复制文件描述符表来建立父子进程间的连接。微内核系统在创建新的线程 / 进程时，内核通常也允许父进程将特定的 Capability 直接继承给子进程。这种继承的方式对于一些不希望暴露到全局的通信连接来说是一个保密性更好的方式，不过它的适用场景相对也比较有限。

8.1.11 总结

图 8.5 给出了宏内核操作系统中常见的进程间通信机制的对比。可以看到，虽然在 IPC 的几个设计角度上这些方案各有异同，但是它们之间的主要区别是在数据抽象上。例如，管道提供字节流的数据抽象，并通过文件抽象（即文件描述符）暴露接口给应用使用，而共享内存则使用内存区间作为数据抽象，并通过内存抽象（访存操作）供应用使用。在实际的应用中，虽然多种 IPC 方案都可以作为通信的选择，但是应用程序往往会根据对于数据抽象的需求来选择具体的方案。后续章节将围绕管道、共享内存和消息队列介绍 IPC 机制。

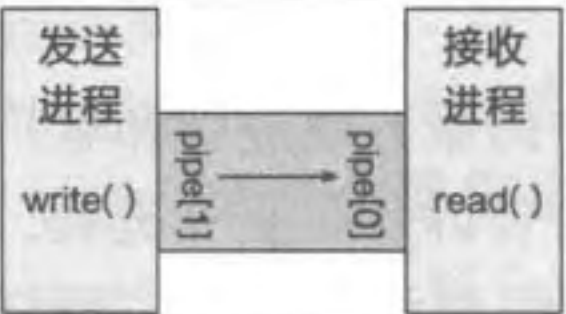
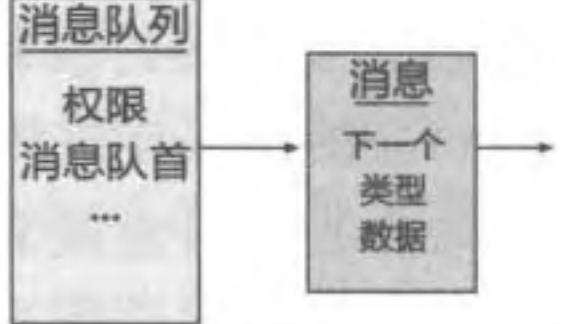
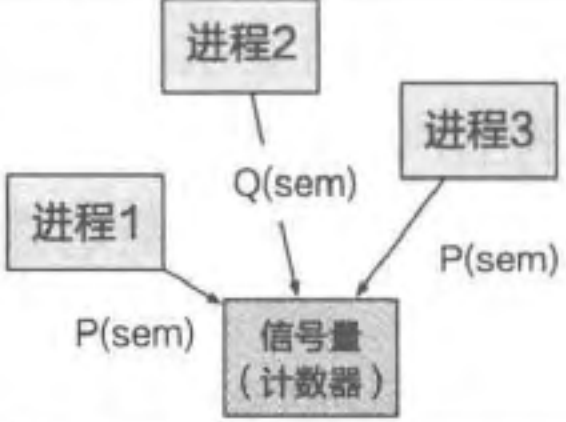
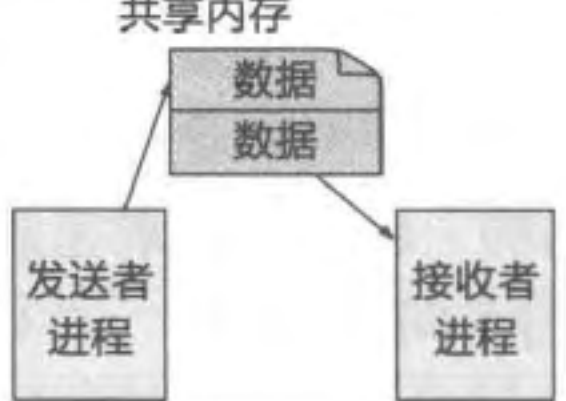
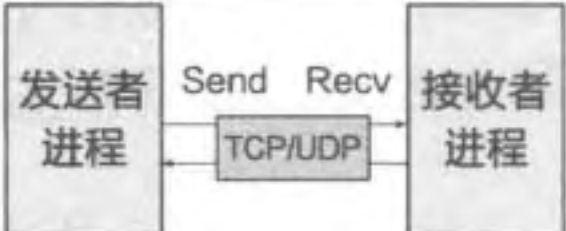
	IPC 机制	数据抽象	参与者	方向	内核实现
	管道	字节流	两个进程	单向	通常以 FIFO 的缓冲区来管理数据。有匿名管道和命名管道两类主要实现
	消息队列	消息	多进程	单向 / 双向	队列的组织方式。通过文件的权限来管理对队列的访问
	信号量	计数器	多进程	单向 / 双向	内核维护共享计数器。通过文件的权限来管理对计数器的访问
	共享内存	内存区间	多进程	单向 / 双向	内核维护共享的内存区间。通过文件的权限来管理对共享内存的访问
	信号	事件编号	多进程	单向	为线程 / 进程维护信号等待队列。通过用户 / 组等权限来管理信号的操作
	套接字	数据报文	两个进程	单向 / 双向	基于 IP / 端口和基于文件路径两种寻址方式。利用网络栈来管理通信

图 8.5 宏内核进程间通信机制的对比

8.2 文件接口 IPC：管道

本节主要知识点

- ❑ 管道进程间通信是如何设计的？
- ❑ Linux 内核中的管道是如何实现的？
- ❑ 命名管道和匿名管道的区别是什么？